

TRAVAUX PRATIQUES

ECOTRACK : Base de Données Production PostgreSQL

ÉCOLE INGETIS PARIS

21 février 2026

Réaliser par : Olivier Ouedraogo et Ferdinand

TRAVAUX PRATIQUES

ECOTRACK : Base de Données Production PostgreSQL

PARTIE 1 : AUDIT & DIAGNOSTIC

Mission 1.1 : État des Lieux de notre Base

Contexte : Compréhension de la taille, le nombre de lignes, les index existants de toutes nos tables de notre Base de données DW.

1. Les 5 plus grosses tables

La requette ci-dessus nous illustre la taille data seule, la taille index seule, la taille totale ainsi que le nombre estimé de ligne de notre base de données DW :

```
SELECT
    schemaname,
    relname AS nom_table,
    pg_size_pretty(pg_total_relation_size(relid)) AS taille_totale,
    pg_size_pretty(pg_relation_size(relid)) AS taille_data,
    pg_size_pretty(pg_indexes_size(relid)) AS taille_index,
    n_live_tup AS nb_lignes_estime
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 5;
```

Data Output Messages Notifications							
Showing rows: 1 to 5 Page No: 1							
	schemaname name	nom_table name	taille_totale text	taille_data text	taille_index text	nb_lignes_estime bigint	
1	dw	fact_measureme...	2268 MB	1544 MB	723 MB	14979802	
2	public	spatial_ref_sys	7144 kB	6896 kB	208 kB	0	
3	dw	dim_time	728 kB	408 kB	288 kB	0	
4	dw	dim_container	632 kB	328 kB	264 kB	0	
5	dw	dim_route	264 kB	104 kB	128 kB	500	

Observations structurelles :

FACT_MEASUREMENT contient 14.9 millions de lignes, utilise 1.5 GB de données avec 723 MB d'index dont la Table principale du Data Warehouse.

Tables de dimension

- Très petites (moins de 1 MB chacune)
- Peu volumineuses

- Faible impact mémoire

En resumer la Structure des tables dimentions sont cohérente avec une architecture en étoile.

Remarque : n_live_tup est une estimation (basée sur stats).

Résultat AVANT ANALYZE

Nous remarquon certaines tables (dim_time, dim_container) affichant :
nb_lignes_estime = 0

Explication

La colonne **n_live_tup** provenant de pg_stat_user_tables :

- n'est PAS un comptage réel
- est une estimation statistique
- est mise à jour uniquement après :
 - o ANALYZE
 - o VACUUM ANALYZE
 - o ou déclenchement d'autovacuum

Résultat APRÈS ANALYZE :

Pour que l'estimations soit cohérentes avec la réalité des tables , ANALYZE scanne l'échantillon des table, calcule des statistique sur la distribution des données et mes à jour pg_stat_user_tables

ci-dessus les 5 plus gros table incluant spatial_ref_sys permettant la pour récupération **des systèmes de coordonnées**.

Data Output Messages Notifications						
	schemaname name	nom_table name	taille_totale text	taille_data text	taille_index text	nb_lignes_estime bigint
1	dw	fact_mesureme...	2268 MB	1544 MB	723 MB	14979802
2	public	spatial_ref_sys	7144 kB	6896 kB	208 kB	8500
3	dw	dim_time	728 kB	408 kB	288 kB	5844
4	dw	dim_container	632 kB	328 kB	264 kB	2001
5	dw	dim_route	264 kB	104 kB	128 kB	500

2.) Taille totale de la base

Notre base de données DW comporte 2286 MB

	taille_base text
1	2286 MB

Requette : **SELECT**

pg_size_pretty(pg_database_size(current_database())) AS taille_base;

3.) Combien d'index par table + type

	schemaname name	tablename name	indexname name	indexdef text
1	dw	dim_agent	dim_agent_pkey	CREATE UNIQUE INDEX dim_agent_pkey ON dw.dim_agent L
2	dw	dim_agent	dim_agent_agent_bk_key	CREATE UNIQUE INDEX dim_agent_agent_bk_key ON dw.din
3	dw	dim_container	idx_dim_container_bk	CREATE INDEX idx_dim_container_bk ON dw.dim_container
4	dw	dim_container	idx_dim_container_geom	CREATE INDEX idx_dim_container_geom ON dw.dim_contair
5	dw	dim_container	idx_dim_container_current	CREATE INDEX idx_dim_container_current ON dw.dim_conta
6	dw	dim_container	dim_container_pkey	CREATE UNIQUE INDEX dim_container_pkey ON dw.dim_cor
7	dw	dim_device	dim_device_pkey	CREATE UNIQUE INDEX dim_device_pkey ON dw.dim_device
8	dw	dim_route	idx_dim_route_geom	CREATE INDEX idx_dim_route_geom ON dw.dim_route USIN
9	dw	dim_route	idx_dim_route_bk	CREATE UNIQUE INDEX idx_dim_route_bk ON dw.dim_route
10	dw	dim_route	dim_route_route_bk_key	CREATE UNIQUE INDEX dim_route_route_bk_key ON dw.dim

	schemaname name	tablename name	indexname name	indexdef text
11	dw	dim_route	dim_route_pkey	CREATE UNIQUE INDEX dim_route_pkey ON dw.dim_route U:
12	dw	dim_time	dim_time_date_bk_key	CREATE UNIQUE INDEX dim_time_date_bk_key ON dw.dim_t
13	dw	dim_time	dim_time_pkey	CREATE UNIQUE INDEX dim_time_pkey ON dw.dim_time USI
14	dw	dim_waste_type	dim_waste_type_pkey	CREATE UNIQUE INDEX dim_waste_type_pkey ON dw.dim_w
15	dw	dim_zone	dim_zone_pkey	CREATE UNIQUE INDEX dim_zone_pkey ON dw.dim_zone US
16	dw	fact_measurement	idx_fact_container_time	CREATE INDEX idx_fact_container_time ON dw.fact_measur
17	dw	fact_measurement	idx_fact_zone	CREATE INDEX idx_fact_zone ON dw.fact_measurement USI
18	dw	fact_measurement	fact_measurement_pkey	CREATE UNIQUE INDEX fact_measurement_pkey ON dw.faci
19	dw	fact_measurement	idx_fact_time	CREATE INDEX idx_fact_time ON dw.fact_measurement USII
20	dw	fact_measurement	idx_fact_container	CREATE INDEX idx_fact_container ON dw.fact_measuremen

Interprétations dans les tableaux ci-dessous

DIM_ROUTE

Type index	btree	
Problème detecté	idx_dim_route_bk	(UNIQUE, btree route_bk)

	dim_route_route_bk_key (UNIQUE, btree route_bk)
Impact	- Double stockage inutile - Double maintenance lors des INSERT - Espace disque gaspillé
Solution	Garde un seul index UNIQUE sur route_bk.

DIM_CONTAINER

Index	Type index
Dim_container_pkey	UNIQUE btree (container_sk)
Ind_dim_container_bk	- btree
Idx_dim_container_geom	GIST
Idx_dim_container_current	Btree (is_current)

Fact_Measurement

Index	Role
Fact_measurement_pkey	PK
Idx_fat_time	- filtre date
Idx_fact_container	Filtre date
Idx_container_time	composite
Idx_fact_zone	Filtre_zone

Problème :

Nous detectons une redondance pour:

idx_fact_container

idx_fact_container_time

idx_fact_container est probablement redondant, vue un index composite existant.

Solution

Supresion recommander pour **'DROP INDEX dw.idx_fact_container;'**

DIM_AGENT, DIM_DEVICE, DIM_WASTE_TYPE, DIM_ZONE

Structure classique :

- **PK sur SK**
- **UNIQUE sur BK**

4.) Tables sans clé primaire

	schema_name name	table_name name
--	---------------------	--------------------

Nous n'en disposons pas de tables sans un clé primaire dans notre base de données DW. Toutes nos tables dimensions et la fact ont une PK.

Requette de verification utiliser ci-dessous :

SELECT

n.nspname AS schema_name,

c.relname AS table_name

FROM pg_class c

JOIN pg_namespace n ON n.oid = c.relnamespace

WHERE c.relkind = 'r'

AND NOT EXISTS (

SELECT 1

FROM pg_constraint con

WHERE con.conrelid = c.oid

AND con.contype = 'p'

)

AND n.nspname NOT IN ('pg_catalog', 'information_schema')

ORDER BY schema_name, table_name;

5.) Type de colonne utilizes

	table_schema name	table_name name	column_name name	data_type character varying	character_maximum_length integer
1	dw	dim_agent	etl_date	timestamp without time zo...	[null]
2	dw	dim_agent	agent_bk	character varying	50
3	dw	dim_agent	firstname	character varying	100
4	dw	dim_agent	lastname	character varying	100
5	dw	dim_agent	role	character varying	50
6	dw	dim_agent	agent_sk	integer	[null]
7	dw	dim_agent	is_active	boolean	[null]
8	dw	dim_container	date_fin	date	[null]
9	dw	dim_container	date_debut	date	[null]
10	dw	dim_container	geom	USER-DEFINED	[null]

	table_schema name	table_name name	column_name name	data_type character varying	character_maximum_length integer
11	dw	dim_container	container_bk	character varying	50
12	dw	dim_container	zone_bk	character varying	50
13	dw	dim_container	status	character varying	20
14	dw	dim_container	container_type	character varying	50
15	dw	dim_container	longitude	double precision	[null]
16	dw	dim_container	container_sk	integer	[null]
17	dw	dim_container	capacity_l	integer	[null]
18	dw	dim_container	latitude	double precision	[null]
19	dw	dim_container	created_at	timestamp without time zo...	[null]
20	dw	dim_container	is_current	boolean	[null]
21	dw	dim container	version	integer	[null]

Role pour chaque type:

Type PostgreSQL	Usage principal dans ton DW	Rôle fonctionnel
integer	Clés étrangères (FK), SK	Jointures, relations entre tables

<code>bigint</code>	Identifiants volumineux, métriques volumineuses	Support gros volumes (fact table)
<code>smallint</code>	Indicateurs numériques faibles	Optimisation espace (ex: flags numériques)
<code>numeric</code>	Mesures précises	Données quantitatives exactes (poids, coûts, ratios)
<code>double precision</code>	Mesures calculées / distances	Performance + calcul scientifique
<code>character varying</code>	Business keys, noms, codes	Données descriptives
<code>text</code>	Descriptions longues	Commentaires ou libellés
<code>boolean</code>	Flags (is_current, actif, etc.)	Indicateurs logiques
<code>date</code>	Dimension temps	Filtrage temporel
<code>timestamp without time zone</code>	Événements internes	Horodatage technique
<code>timestamp with time zone</code>	Données temps réel	Synchronisation multi-zones
<code>USER-DEFINED</code>	GEOMETRY (PostGIS)	Données spatiales (POINT, LINESTRING)
<code>name</code>	Métadonnées système	Colonnes internes PostgreSQL
<code>oid</code>	Objets internes	Système PostgreSQL

Requêtes:

SELECT

`table_schema,`

`table_name,`

`column_name,`

`data_type,`

`character_maximum_length`

FROM `information_schema.columns`

WHERE `table_schema NOT IN ('pg_catalog', 'information_schema')`

ORDER BY `table_name;`

MISSION 1.2 — REQUÊTES LENTES

Activation pg_stat_statements par “**CREATE EXTENSION IF NOT EXISTS pg_stat_statements;**” pour les requette lentes

	query text
1	select * from DW.fact_Measurement
2	INSERT INTO DW.FACT_MEASUREMENT (measurement_timestamp, time_sk, container_sk, zone_sk, taux_remplissage_pct,
3	select * from DW.fact_Measurement
4	CREATE INDEX IF NOT EXISTS idx_fact_container_time
5	CREATE INDEX IF NOT EXISTS idx_fact_zone
6	CREATE INDEX IF NOT EXISTS idx_fact_container
7	CREATE INDEX IF NOT EXISTS idx_fact_time
8	/*pga4dash*/ SELECT \$1 AS chart_name, pg_catalog.row_to_json(t) AS chart_data FROM (SELECT (SELECT count(*) FROM pg_catalog.pg_stat_activity WHERE c
9	CREATE INDEX IF NOT EXISTS idx_dim_route_geom
10	select * from DW.dim_container

Suite vertical ci-dessus

	calls bigint	total_exec_time double precision	mean_exec_time double precision	rows bigint
	1	516294.6901	516294.6901	182500000
volume_litres, ...	1500	354183.4795999997	236.1223197333337	15000000
	3	50949.971399999995	16983.3238000000002	31500000
	1	12593.1741	12593.1741	0
	1	11246.3053	11246.3053	0
	1	11084.9366	11084.9366	0
	1	10029.4875	10029.4875	0
datname = (SELECT datname FROM pg_catalog.p...	1612	910.1847000000001	0.5646307071960299	8060
	1	703.5076	703.5076	0
	4	562.6930999999998	140.67327499999996	8004

Signification des colonnes

Colonne	Signification
nb_appels	Nombre de fois que la requête a été exécutée
temps_total	Temps cumulé total en millisecondes
temps_moyen	Temps moyen par exécution
nb_lignes	Nombre total de lignes traitées/retournées
Requête la plus lourde	select * from DW.fact_measurement 1 516294 ms 516294 ms 182 500 000 lignes

Colonne	Signification
Interprétations	Exécutée 1 fois
	A pris 516 secondes (~8.6 minutes)
	A retourné 182 millions de lignes

Résumer :

Les requêtes lentes observées proviennent principalement de scans complets sur la table de faits (fact_measurement).

Ces requêtes parcourent l'intégralité des données sans filtre, ce qui entraîne un temps d'exécution très élevé proportionnel au volume de données.

Les opérations d'insertion batch présentent un temps total élevé dû au volume massif traité, mais un temps moyen par exécution satisfaisant.

La création des index a un coût initial important mais nécessaire pour optimiser les futures requêtes analytiques.

REQUETTE :

SELECT

query,

calls,

total_exec_time,

mean_exec_time,

rows

FROM pg_stat_statements

ORDER BY total_exec_time DESC

LIMIT 10;

MISSION 1.3 — PROJECTION DE CROISSANCE

1.) Nombre de lignes sur 1 an

Le nombre de lignes sur 1 an dans notre base nous fait :

$2000 \text{ containers} \times 500\,000 \text{ mesures/jour} \times 365$

Resultat :

182 500 000 ligne

2 .) Taille estimée

Une ligne $\approx$ 80 bytes (moyenne réaliste avec index).

$182\,500\,000 \times 80 \text{ bytes}$

$= 14\,600\,000\,000 \text{ bytes}$

$\approx 13.6 \text{ GB}$

PARTIE 2 : OPTIMISATION PERFORMANCE

Objectif : passer de requêtes lentes à **10x–100x plus rapides** grâce à :

- Indexation intelligente
- Partitionnement
- Configuration mémoire

Mission 2.1 – Indexation Stratégique

1.) Quand utiliser un index B-tree ?

B-tree est un index par défaut dans PostgreSQL. Elle Optimise tri + filtre et
Très rapide pour égalité et comparaisons

On l'utilise pour des requettes GROUP BY, ORDER BY, Jointure (JOIN ON id=...), des colonnes (integer, biging, date ...), des Conditions BETWEEN, < , <=, >, >=).

Cas d'usage pratique pour une Analyse temporelle et Analyse par conteneur :

Requette :

SELECT *

SELECT *

FROM DW.FACT_MEASUREMENT

WHERE measurement_timestamp BETWEEN '2025-05-01' AND '2025-05-31';

	measurement_timestamp timestamp without time zone	taux_replissage_pct numeric (5,2)	volume_litres numeric (9,2)	temperature_c numeric (5,2)	batterie_pct smallint	po nu
1	2025-05-01 19:35:49	56.29	1500.96	0.23	64	
2	2025-05-01 21:48:45	22.93	498.65	0.19	99	
3	2025-05-01 12:32:13	95.76	1578.10	0.35	80	
4	2025-05-01 02:39:37	0.68	1182.25	32.10	71	
5	2025-05-01 23:11:33	63.07	73.48	14.00	84	
6	2025-05-01 04:30:54	90.18	1815.55	-0.27	77	
7	2025-05-01 16:01:15	36.63	1585.33	0.02	59	
8	2025-05-02 22:13:36	52.85	1142.58	-0.78	14	
9	2025-05-02 16:11:08	46.21	1351.18	0.76	48	
Total rows: 15000005 Querv complete 00:00:40.874 CRLF Ln 4. Col 1						

Requette:

SELECT AVG(taux_replissage_pct)

FROM DW.FACT_MEASUREMENT

WHERE container_sk = 120

AND measurement_timestamp >= '2025-05-01';

	avg numeric	🔒
1	50.0946545439708402	

Total rows: 1	Query complete 00:00:27.655
---------------	-----------------------------

2.) Quand utiliser un index GIN ?

GIN = Generalized Inverted Index

On utilise GIN pour les colonnes JSONB, faire les recherches dans des tableaux (ARRAY), texte (tsvector) et aussi les opérateurs (@>, ?, ?|)

GIN est plus lourd en écriture, elle est utilisée uniquement sur les requêtes JSON contrairement au format de notre base de données.

3.) Quand utiliser un index BRIN ?

BRIN = Block Range Index

On utilise **BRIN** pour les tables très volumineuses (Fact_Measurement), les Données naturellement triées, les colonnes timestamp, date, bigint, les Tables de logs etc.

BRIN est idéal pour les tables > 1M lignes et elle est beaucoup plus rapide qu'un Seq Scan

MISSION : Optimisation requête #1 identifiée en Partie 1

Analytique typique : Cas d'usage sur la table fact

Requête :

```
SELECT zone_sk,
       AVG(taux_remplissage_pct)
FROM DW.FACT_MEASUREMENT
```

WHERE measurement_timestamp BETWEEN '2025-05-01' AND '2025-05-31'
GROUP BY zone_sk;

	zone_sk integer	avg numeric
1	1	50.1329132591500201
2	3	49.7319366173390094
3	4	49.9043294172775044
4	5	49.8150155847675837
5	6	49.5586631509558339
6	7	50.3515180114316097
7	8	50.2512231845436376
8	9	49.7709112919496940
9	10	49.9614654415242213

Y a-t-il un WHERE ?	Y a-t-il un GROUP BY ?	Index simple ou composite ?
Oui : measurement_timestamp BETWEEN ... Colonne candidate pour index.	Oui : GROUP BY zone_sk index nous aider si bien structuré.	WHERE measurement_timestamp BETWEEN GROUP BY zone_sk L'ordre logique : Filtrer par date Puis agréger par zone

Exemple Index composite:

CREATE INDEX idx_fact_container_timestamp
ON DW.FACT_MEASUREMENT (container_sk, measurement_timestamp);

CREATE INDEX IF NOT EXISTS idx_fact_container_time
ON DW.FACT_MEASUREMENT(container_sk, time_sk);

4. Choix entre d'index : B-tree, GIN, BRIN ?

BRIN sur measurement_timestamp : table volumineuse, données temporelles insérées chronologiquement

Avantage : index très léger, optimise les scans par plage

Requête

```
CREATE INDEX idx_fact_timestamp_brin  
ON DW.FACT_MEASUREMENT  
USING BRIN (measurement_timestamp);
```

```
CREATE INDEX
```

```
Query returned successfully in 1 min 35 secs.
```

```
EXPLAIN ANALYZE
```

```
SELECT zone_sk,  
       AVG(taux_remplissage_pct)  
FROM DW.FACT_MEASUREMENT  
WHERE measurement_timestamp  
       BETWEEN '2025-05-01' AND '2025-05-31'  
GROUP BY zone_sk;
```

	QUERY PLAN text
1	Finalize GroupAggregate (cost=3568646.83..3568867.77 rows=847 width=36) (actual time=19830.308..19852.597 rows=84
2	Group Key: zone_sk
3	-> Gather Merge (cost=3568646.83..3568844.47 rows=1694 width=36) (actual time=19830.280..19849.928 rows=2541 loc
4	Workers Planned: 2
5	Workers Launched: 2
6	-> Sort (cost=3567646.80..3567648.92 rows=847 width=36) (actual time=19756.190..19756.279 rows=847 loops=3)
7	Sort Key: zone_sk
8	Sort Method: quicksort Memory: 104kB
9	Worker 0: Sort Method: quicksort Memory: 104kB

15	Worker 1: Batches: 1 Memory Usage: 553KB
16	-> Parallel Bitmap Heap Scan on fact_measurement (cost=4155.32..3535729.07 rows=6373192 width=10) (actu
17	Recheck Cond: ((measurement_timestamp >= '2025-05-01 00:00:00':timestamp without time zone) AND (mea
18	Rows Removed by Index Recheck: 118507
19	Heap Blocks: lossy=67647
20	-> Bitmap Index Scan on idx_fact_timestamp_brin (cost=0.00..331.40 rows=15296715 width=0) (actual time=
21	Index Cond: ((measurement_timestamp >= '2025-05-01 00:00:00':timestamp without time zone) AND (mea
22	Planning Time: 54.767 ms
23	Execution Time: 19854.539 ms

L'index BRIN est utilisé et réduit le scan global

Bitmap Heap Scan

-> Bitmap Index Scan using idx_fact_timestamp_brin

Tableau comparative:

Index testé	EXPLAIN avant index	EXPLAIN après index	
Nom : idx_fact_timestamp_brin Type : BRIN Colonne : measurement_timestamp	Seq Scan sur 129M lignes Temps estimé > 60 secondes	Bitmap Index Scan utilisé Parallel Bitmap Heap Scan ~15M lignes scannées Execution Time : 19.8s	

En resumer :

- L'index est correctement utilisé
- La requête reste coûteuse car elle traite un ans complet (~180M lignes)
- BRIN réduit le scan global mais ne peut pas éviter le traitement des lignes correspondant au filtre

Mission 2.3 : Optimisation Mémoire PostgreSQL

Contexte : PostgreSQL a de nombreux paramètres de performance. Les plus critiques : `shared_buffers`, `work_mem`, `effective_cache_size`.

MISSION : Analyser la configuration actuelle

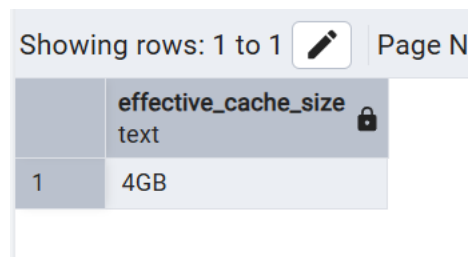
1.)Comment voir la configuration PostgreSQL actuelle ?

Requête :

`SHOW shared_buffers;`

`SHOW work_mem;`

`SHOW effective_cache_size;`



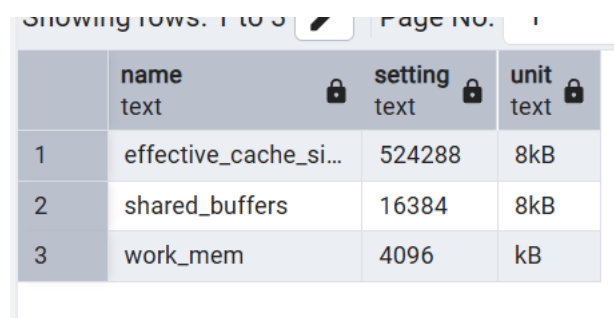
Showing rows: 1 to 1 | Page N

	effective_cache_size
1	4GB

`SELECT name, setting, unit`

`FROM pg_settings`

`WHERE name IN ('shared_buffers','work_mem','effective_cache_size');`



Showing rows: 1 to 3 | Page No. 1

	name	setting	unit
1	effective_cache_size	524288	8kB
2	shared_buffers	16384	8kB
3	work_mem	4096	kB

2.)INTERPRETATIONS :

Rôle **shared_buffers**:

Mémoire allouée au cache interne PostgreSQL.

PostgreSQL garde pages de table, index et blocs récemment lus

Rôle `work_mem` :

Mémoire allouée **par opération** :

- GROUP BY
- ORDER BY
- Hash Join
- Hash Aggregate

Rôle `effective_cache_size` :

Indique au planner combien de mémoire totale (OS + PostgreSQL) est disponible pour le cache.

3.) Quelle est la valeur de `work_mem` ?

Valeur actuelle :

4 MB

Interprétation

Avec **4 MB** :

- Suffisant pour des agrégations modestes
- Peut devenir limitant pour de gros tris ou jointures
- Trop faible **avec** tri sur disque